

Advanced Prompt Engineering

Koustav Rudra
Assistant Professor
AI, IIT Kharagpur



20/09/2026

What We'll Cover

- **Foundations**
 - Intro to Prompting
 - Zero-Shot Prompting
 - Few-Shot Prompting
- **Reasoning Techniques**
 - Chain-of-Thought (CoT)
 - Self-Consistency
 - Tree-of-Thought (ToT)
- **Acting & Tool Use**
 - ReAct
- **Steering the Model**
 - Meta Prompting
 - Directional Stimulus Prompting

Introduction

PROMPT



*Programming with
natural language*

Because you are:

- Giving instructions
- Specifying constraints
- Defining inputs and outputs
- Controlling behavior
- Iterating to improve results

Example:

Traditional code:

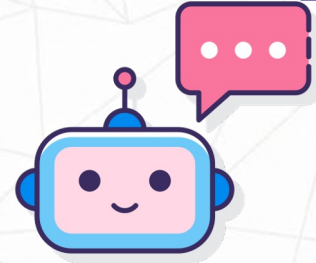
```
sort(numbers, descending=True)
```

Prompt-based instruction:

```
Sort these numbers from largest to  
smallest and explain the steps.
```

Introduction: Prompting

Put an input into a format (a “prompt”) that encourages the model to carry out a task.



x : “I love advanced NLP!”

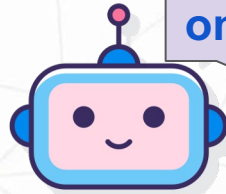
$$p_{\theta}(y \mid \underbrace{f(x)}_{\text{Prompt}})$$

$f(x)$: Classify this sentence’s sentiment as either Positive or Negative: “I love advanced NLP!”

Example y : The sentence is positive.

Introduction: Autoregressive language model

An autoregressive language model is an AI model that generates text one **token** at a time, always **predicting the next token** based on everything that came before it.



The cat sat
on the ...

A **"token"** can be:

- a word
- part of a word
- punctuation
- or even spaces/symbols

The probability of a whole sentence is built by repeatedly predicting the next token from previous tokens.

The model predicts probabilities like:

- mat → 62%
- floor → 15%
- chair → 8%
- roof → 3%

So generation looks like:

"The" → "cat" → "sat" → "on" → "the" → "mat"

What is a Prompt?

- Prompt is initial text inputs given to a model, which it uses to generate responses or accomplish tasks.
- A prompt serves as an interaction point between a user and an LLM, allowing the model to generate specific outputs based on the input provided.
- It can take various forms, including questions, commands, text snippets, or even code, and can range from single words to complex paragraphs.

What is a Language Model?

A question

Write me a story about full moon

A command

Classify the following sentence on basis of Positive or negative sentiment:
Sentence: What a beautiful day

A prompt with text snippet

What is a Prompt?

Write a python function that takes two numbers as input, and returns sum of the two numbers

Code Generation

```
"""def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)"""  
Convert this to C++
```

Code Conversion

Prompt: Summarize the following text: 'The quick brown fox jumps over the lazy dog.'

Response: A fox jumps over a dog.

Summarization

Prompt: Recommend a new book for a mystery lover.

Response: I recommend 'The Silent Patient' by Alex Michaelides.

Personalization

Prompt: Evaluate the sentiment of the following review: 'This product exceeded my expectations!'

Response: Positive sentiment

Evaluation

What are the different prompt types?

- For LLMs, we can generally define two types :
 - **system prompts** i.e. providing clear guidelines for the model on how to respond to user inputs.
 - **user prompts** i.e. inputs by end-users to interact with the AI system and obtain desired outputs.
- The **effectiveness of a prompt** directly influences the **relevance** and **accuracy** of the model's response.
- **Well-structured prompts** help the model understand the user's intent better.

What is Prompt Engineering?



- The 'engineering' term in *Prompt Engineering* is used as it involves systematic prompt design process, optimized for the tasks at hand that is achieved through iterative refinement.
- Carefully crafted prompts help to understand users' intent better.

Common LLM inference terms



Tokens

Temperature

Top k

Top p (nucleus sampling)

Common LLM inference terms - tokens

- **Tokens :** They are the basic units of text that LLMs process. They can represent words, subwords, or characters, depending on the tokenizer used

Pseudohypoparathyroidism and deoxyribonucleic acid methylation affect mitochondrial dysfunction in immunocompromised patients undergoing chemotherapeutic treatment.

Pseudohypoparathyroidism and deoxyribonucleic acid methylation affect mitochondrial dysfunction in immunocompromised patients undergoing chemotherapeutic treatment.

33 tokens

20.00 tokens / 100 chars

The transformer-based model achieved 94.7% accuracy on 1,000 test cases, outperforming GPT-2 by ~3.2x.

The transformer-based model achieved 94.7% accuracy on 1,000 test cases, outperforming GPT-2 by ~3.2x.

31 tokens

30.39 tokens / 100 chars

Subword Tokenization :

Goal: Represent text efficiently while handling both common and rare words.

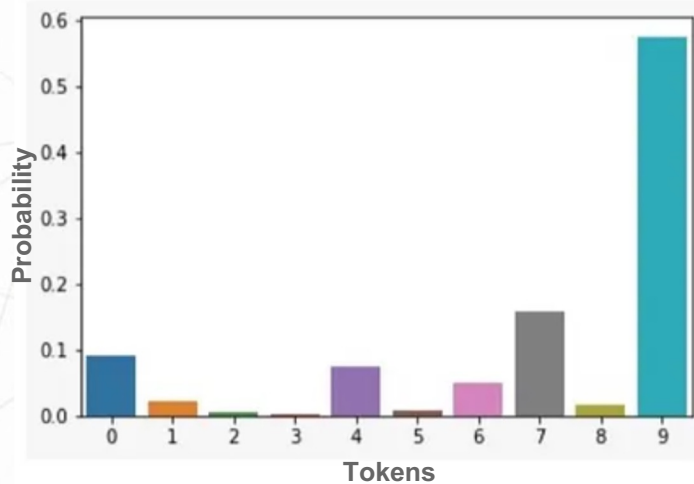
Principle:

- Frequent words remain whole (e.g., "transformer").
- Rare words are split into meaningful subwords (e.g., "methylation" becomes "methyl" and "ation").

Common LLM inference terms - temperature

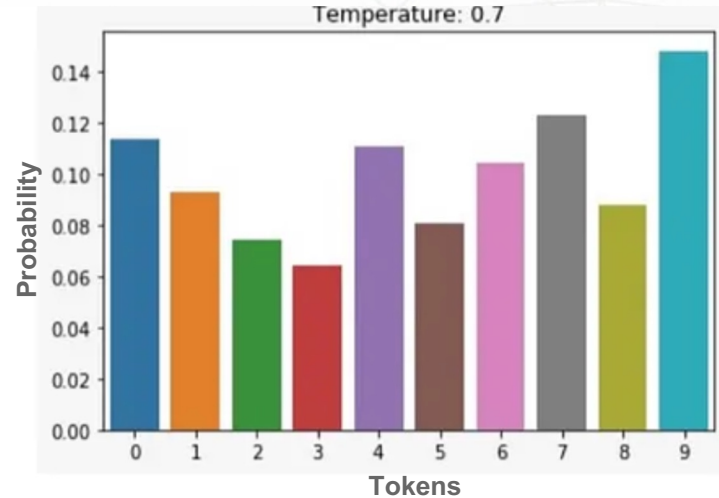
Temperature

- Controls sharpness of the next-token distribution
- Any positive value
- Lower temperature $\rightarrow$ sharper distribution $\rightarrow$ repetitive generations



LESS ENTROPY

INCREASE IN ENTROPY
WITH INCREASE IN T



MORE ENTROPY

Common LLM inference terms - temperature

Temperature scaling in softmax

$$\frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Example :-

Token	Logit
"cat"	4.0
"dog"	2.0
"fish"	1.0
"banana"	0.0

T=0.5

Token	Prob
"cat"	0.979
"dog"	0.018
"fish"	0.002
"banana"	0.0003

Entropy =
 $0.979 \ln(0.979) +$
 $0.018 \ln(0.018) +$
 $0.002 \ln(0.002) +$
 $0.0003 \ln(0.0003)$
= 0.110

T=1

Token	Prob
"cat"	0.831
"dog"	0.112
"fish"	0.041
"banana"	0.015

Entropy =
 $0.831 \ln(0.831) +$
 $0.112 \ln(0.112) +$
 $0.041 \ln(0.041) +$
 $0.015 \ln(0.015)$
= 0.594

T=2

Token	Prob
"cat"	0.579
"dog"	0.213
"fish"	0.129
"banana"	0.078

Entropy =
 $0.579 \ln(0.579) +$
 $0.213 \ln(0.213) +$
 $0.129 \ln(0.129) +$
 $0.078 \ln(0.078)$
= 1.109

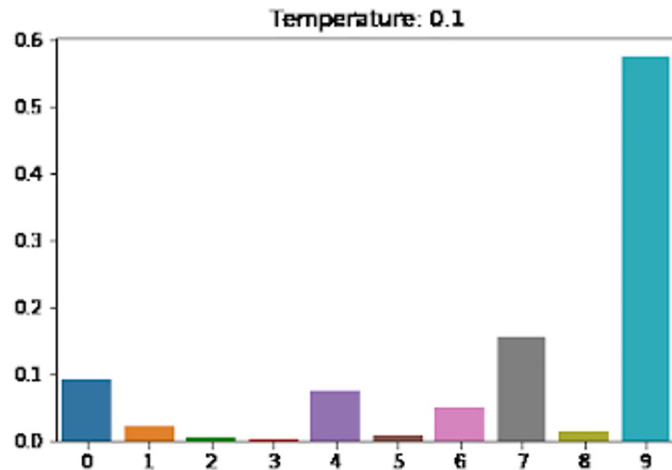
Common LLM inference terms - temperature

Temperature scaling in softmax

$$\frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Temperature (mathematical intuition)

- Scales the logits of the tokens
- **Temp = 1** → softmax uses the same as predicted logits
- **Temp >> 1** → predicted logits have less effect on softmax output
- *Food for thought: What happens if $T = 0$?*



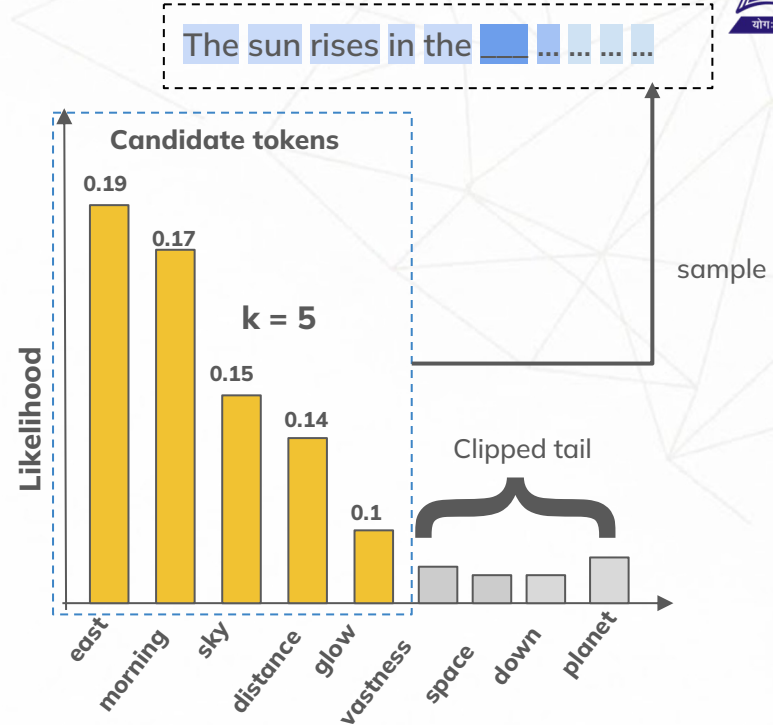
Temp = 0 is not feasible.

In such cases:

- Temp is thresholded to a low value like 0.000001
- Sampling is done by greedy decoding

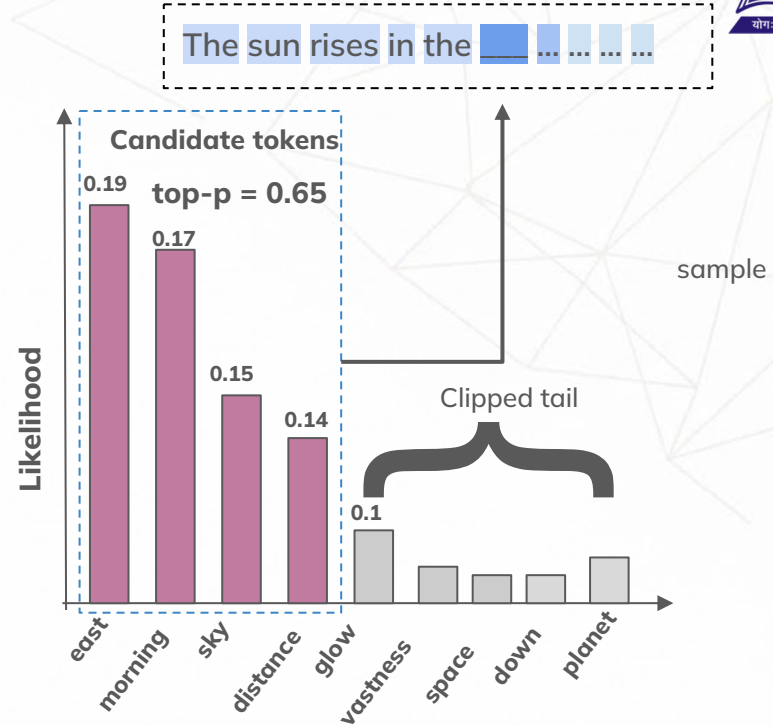
Common LLM inference terms - top k

- **Top_k** : Top-K sampling involves **selecting the top K most likely words** from the probability distribution and then **sampling the next word only from this subset**
- For example, Imagine a scenario where an LLM is prompted with *"The sun rises in the"*.
- The model might assign the highest probabilities to words like "east", "morning", "sky", etc
- With Top-K sampling where $K=5$, the model would only consider the top 5 words with the highest probability for its next output.
- This method **reduces the chances of generating highly improbable words** and **provides a more controlled and predictable output**.



Common LLM inference terms - top p

- **Top_p / Nucleus Sampling** : Instead of selecting a fixed number of top words, the model **selects a subset of words** that **have a cumulative probability exceeding a certain threshold P**.
- For example, Consider the same scenario where an LLM is prompted with “*The sun rises in the*” and $top-p = 0.65$
- If the top 4 words in the probability distribution cumulatively account for 65% of the probability mass and we set $P=0.65$, then the model would sample the next word from this set of 4 words
- Top-p is more dynamic than Top-K, adapting the subset size based on the probability distribution
- It can **capture more diversity** in outputs, **reducing repetitiveness**



Types of Prompting: Overview



Advanced Patterns

ReAct, ToT, Meta-Prompting



Chain-of-Thought

Explicit reasoning



Few-Shot Prompting

Learning from examples



Zero-Shot Prompting

Simple, direct instructions

Zero-Shot Prompting



User : Classify the text into neutral, negative or positive. Just give me the classification.

Text : I think the vacation is okay.

Sentiment :

Mistral : Neutral

- **Direct instructions:** Users tell the AI what to do without providing specific examples.
- **Relies on pre-trained knowledge:** LLMs learn to perform tasks from massive datasets, enabling them to generalize to new, unseen instruction
- **Versatile and adaptable:** Zero-shot prompting makes AI models useful for a wide range of applications without task-specific fine-tuning.

Zero-Shot Prompting

- **Model size matters:** Larger LLMs (e.g. Mistral 123B) are better at zero-shot tasks than smaller models (e.g. Mistral 7B)
- **Complexity challenges:** Zero-shot prompting can struggle with complex tasks where implicit knowledge or specific examples are needed.

Lack of context can lead to:

- Vague or imprecise responses.
- Difficulty in understanding nuanced instructions.

Solution: Providing examples (few-shot prompting) can significantly boost performance on challenging tasks.

Scenario: A server firm deploys an LLM to segregate customer support queries and error reports.

Categories:

User_Error: No fault in system - user needs guidance

Docs_Gap: Missing details in documentation

Bug: Genuine technical fault - needs attention

Billing: No technical error - error in billing system.

User: Classify this support ticket into a category (User_Error, Docs_Gap, Bug, Billing) and severity (Low, Medium, High).

Ticket: "I couldn't figure out how to export my report, wasted an hour."

LLM: Category: User_Error, Severity: Low

Wrong

Few-Shot Prompting

User : This is awesome // Positive
This is bad! // Negative
Wow that movie was rad! // Positive
What a horrible show! //

Mistral : Negative

- Few-shot learning involves providing the model with a small number of high-quality examples that include both input and desired output for the target task.

Benefits : These examples help the model grasp:

1. User intent
2. Desired output format and style

Takeaway: Few-shot learning generally leads to more accurate and relevant responses compared to zero-shot prompting.

Few-Shot Prompting



User: Classify each ticket into a category (User_Error, Docs_Gap, Bug, Billing) and severity (Low, Medium, High), following our conventions.

Ticket: "The export button throws a 500 error every time."

Category: Bug, **Severity:** High

Ticket: "I didn't know the app could schedule reports. Found it by accident."

Category: Docs_Gap, **Severity:** Medium

Ticket: "My last two invoices show the wrong plan and I may have lost saved drafts."

Category: Billing, **Severity:** High

Ticket: "I couldn't figure out how to export my report. Wasted an hour."

Answer: **Category:** Docs_Gap, **Severity:** High

Few-Shot Prompting

- When dealing with more complex reasoning tasks e.g. mathematical reasoning, this method may fail to deliver the correct output
- In this example, we can check that it is not the correct response
- It seems like few-shot prompting is not enough to get reliable responses for this type of reasoning problem

User : Permutations of $\{1,2,3,4,5\}$ are written in one-line notation: the list $[a,b,c,d,e]$ sends $1 \rightarrow a$, $2 \rightarrow b$, $3 \rightarrow c$, $4 \rightarrow d$, $5 \rightarrow e$. You are given a starting number and a sequence of permutations. Apply them in order (first listed is applied first) and report the final position.

Q: Start at 3. Apply $[2,1,4,3,5]$ then $[4,3,2,5,1]$ then $[2,4,5,1,3]$

A: 3

Q: Start at 4. Apply $[2,5,3,4,1]$ then $[3,4,2,1,5]$ then $[3,5,2,1,4]$

A: 3

Q: Start at 3. Apply $[5,1,3,4,2]$ then $[3,4,2,5,1]$ then $[4,3,1,2,5]$ then $[2,3,1,5,4]$ then $[2,3,5,4,1]$ then $[4,5,3,1,2]$ then $[4,3,5,2,1]$ then $[2,5,1,4,3]$.

A:

A: 3

WRONG

Chain-of-Thought (CoT) Prompting

Without CoT

Q: Complex math problem

A: [Direct answer, sometimes wrong]

With CoT

Q: Complex math problem. Let's think step by step

A: Let's break this down:

- *Step 1: [reasoning]*
- *Step 2: [reasoning]*
- *Step 3: [reasoning]*
- *Answer: [correct answer]*

- **Technique:** Guides the LLM to solve complex tasks by breaking them down into smaller, logical steps.
- **How it works:** The prompt encourages the model to "think out loud" and show its reasoning process, leading to more accurate solutions.
- **Effectiveness:** Especially powerful when combined with few-shot prompting, providing both examples and a structured approach.

Chain-of-Thought (CoT) Prompting

User : Permutations of $\{1,2,3,4,5\}$ are written in one-line notation: the list $[a,b,c,d,e]$ sends $1 \rightarrow a$, $2 \rightarrow b$, $3 \rightarrow c$, $4 \rightarrow d$, $5 \rightarrow e$.

You are given a starting number and a sequence of permutations. Apply them in order (first listed is applied first) and report the final position. Perform the permutations step by step and show the steps.

Q: Start at 3. Apply $[2,1,4,3,5]$ then $[4,3,2,5,1]$ then $[2,4,5,1,3]$.

A: $[1,2,3,4,5]$ permuted by $[2,1,4,3,5]$ becomes $[2,1,4,3,5]$. Permuted by $[4,3,2,5,1]$ becomes $[5,4,1,2,3]$. Permuted by $[2,4,5,1,3]$ becomes $[2,5,3,4,1]$. Final position is 3.

Q: Start at 4. Apply $[2,5,3,4,1]$ then $[3,4,2,1,5]$ then $[3,5,2,1,4]$.

A: $[1,2,3,4,5]$ permuted by $[2,5,3,4,1]$ becomes $[5,1,3,4,2]$. Permuted by $[3,4,2,1,5]$ becomes $[4,3,5,1,2]$. Permuted by $[3,5,2,1,4]$ becomes $[1,5,4,3,2]$. Final position is 3.

Q: Start at 3. Apply $[5,1,3,4,2]$ then $[3,4,2,5,1]$ then $[4,3,1,2,5]$ then $[2,3,1,5,4]$ then $[2,3,5,4,1]$ then $[4,5,3,1,2]$ then $[4,3,5,2,1]$ then $[2,5,1,4,3]$.

Chain-of-Thought (CoT) Prompting

A: [1,2,3,4,5] permuted by [5,1,3,4,2] becomes [2,5,3,4,1].
Permuted by [3,4,2,5,1] becomes [1,3,2,5,4].
Permuted by [4,3,1,2,5] becomes [2,5,3,1,4].
Permuted by [2,3,1,5,4] becomes [3,2,5,4,1].
Permuted by [2,3,5,4,1] becomes [1,3,2,4,5].
Permuted by [4,5,3,1,2] becomes [4,5,2,1,3].
Permuted by [4,3,5,2,1] becomes [3,1,5,4,2].
Permuted by [2,5,1,4,3] becomes [5,3,2,4,1].
Final position is 2.



- **Model Size & Complexity:** Larger LLMs are generally better at Chain-of-Thought reasoning.
- CoT prompting can elicit correct answers to complex questions (as shown above), when few-shot prompting fails.
- **Key Takeaway:** Sophisticated prompt engineering techniques like CoT can significantly enhance the reasoning capabilities of existing LLMs.

Variants of CoT Prompting - Zero-shot CoT

Zero Shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer is

Output: 8



Zero Shot CoT

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: Let's think step by step

Output: There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls.



Simplifying CoT: No need to provide explicit reasoning steps in the prompt.

How it works:

- Add "Let's think step by step" to the original prompt.
- The LLM generates its own reasoning chain and uses it to arrive at the answer.

Benefits:

- **Task-agnostic:** Applicable to a wide range of tasks without needing task-specific prompts.
- **Encourages deeper reasoning:** Promotes more thoughtful and structured problem-solving by the LLM.

Variants of CoT Prompting - Few-shot CoT

Few Shot

Model Input

Q. Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can have 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11

Q. The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A : The answer is 27



Few Shot CoT

Model Input

Q. Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can have 3 tennis balls. How many tennis balls does he have now?

A : Roger started with 5 balls, 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11

Q. The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

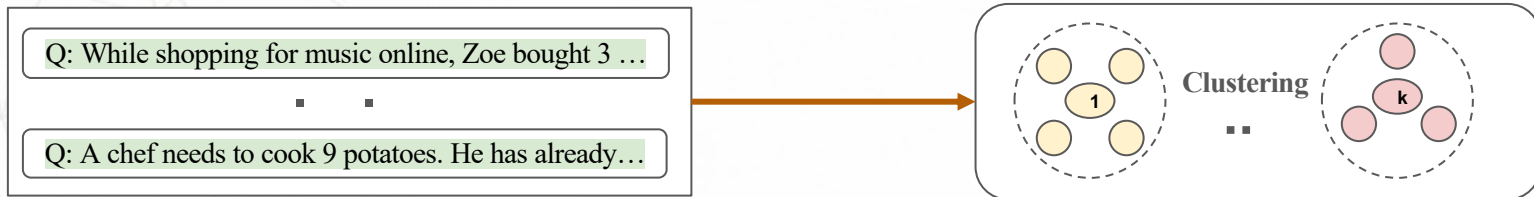
Model Output

The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9



Variants of CoT Prompting - Automatic CoT

- **Goal:** Automate the creation of reasoning chains, reducing reliance on manually crafted examples.
- **Two-Stage Process:**
 1. **Question Clustering:** Questions from a dataset are grouped into clusters based on similarity. This helps in identifying representative questions for each cluster, ensuring diversity in the examples generated.

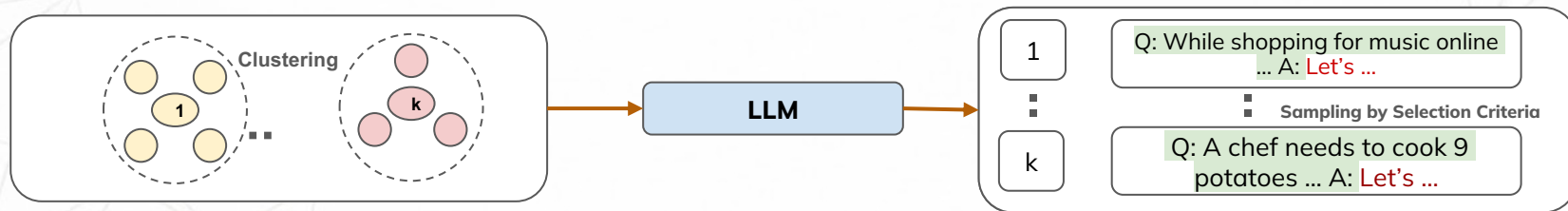


Variants of CoT Prompting - Automatic CoT



2. Demonstration Sampling:

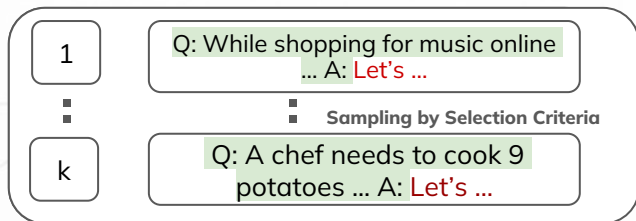
2.a A representative question is chosen from each cluster.



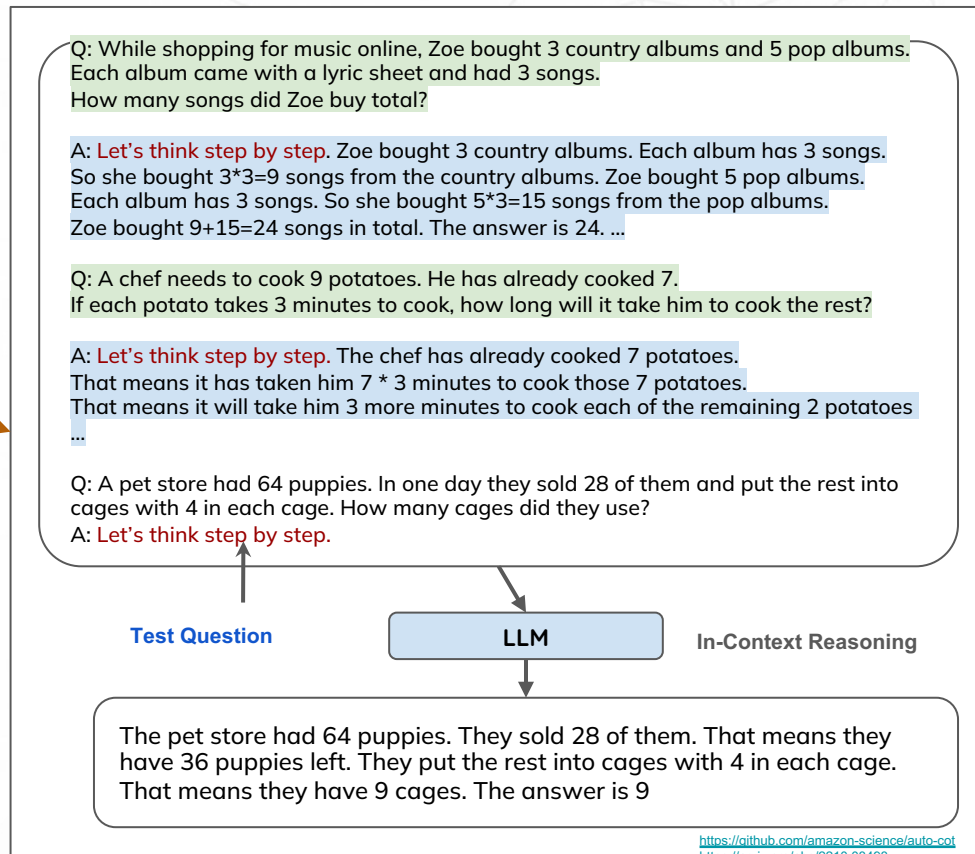
Variants of CoT Prompting - Automatic CoT



2.b Zero-Shot-CoT (with heuristics) generates a reasoning chain for the chosen question.



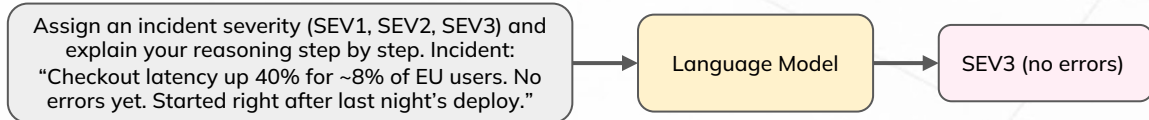
Outcome: Auto-CoT achieves comparable or superior performance to traditional CoT and few-shot prompting, especially for reasoning tasks.



CoT Prompting - Self Consistency

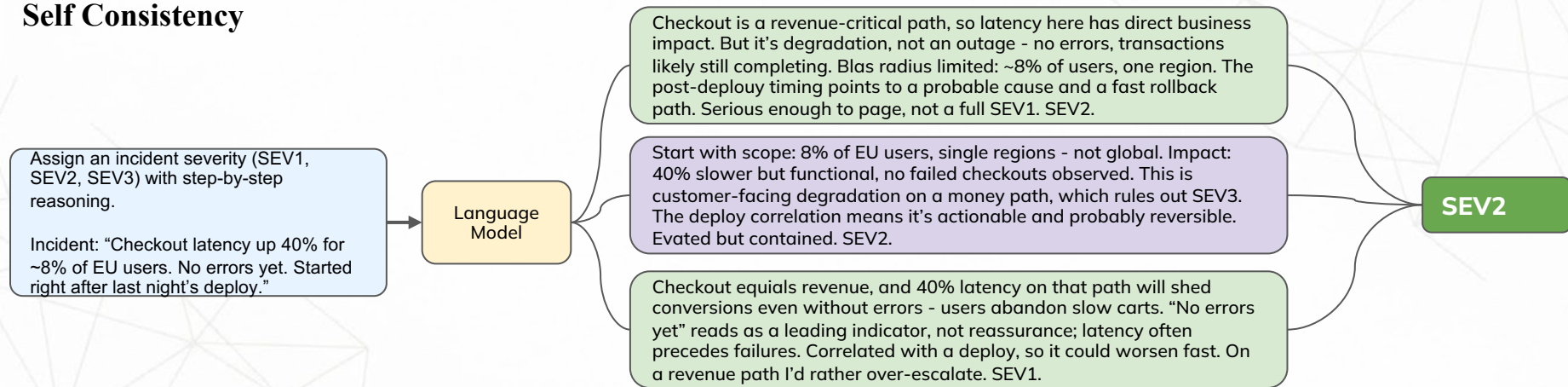


Chain-of-Thought Prompting



Greedy Decode

Self Consistency



- Self-consistency leverages the intuition that a complex reasoning problem typically admits multiple different ways of thinking leading to its unique correct answer
- It first samples a diverse set of reasoning paths instead of only taking the greedy one, and then selects the most consistent answer by marginalizing out the sampled reasoning paths.

ReAct Prompting

A framework where LLMs generate reasoning traces and task-specific actions in an interleaved manner.

The model doesn't just think — it acts.

It calls tools, searches the web, and queries APIs, then uses what it observes to refine the next thought.

ReAct is inspired by the synergies between “**acting**” and “**reasoning**”, the same way humans learn new tasks and make decisions or reasoning.

The Core Loop

Question: Which river flows through the city that hosts the Taj Mahal, and where does it originate?



THOUGHT

Reason about what to do next

“I need to find which city has the Taj Mahal, then find the river flowing through it, then its origin.”



ACTION

Call a tool or take an action

Search [Taj Mahal location]



OBSERVATION

Feed the result back into the next thought

The Taj Mahal is located in Agra, on the banks of the Yamuna river.

*🔄 repeats until **Finish [answer]***

ReAct Prompting

In the example (dynamic question answering), we pass a prompt like the following :

Setting: An LLM is used to query from a system of record (CRM/ERP/billing)

Is Acme Corp past due? If so, what is the outstanding amount and who is the account manager?

ITERATION 1 :

ReAct (Reason + Act)

Thought 1: I need to know the billing status first.

Act 1: `query_billing` [account = Acme Corp]

Obs 1: Acme Corp is **past due** with an outstanding amount of ₹1,20,000 for 14 days ...

Process:

- **Thought:** LLM generates an initial thought based on the prompt (e.g., "Thought 1").
- **Act:** The LLM takes an action, often using external tools to gather information (e.g., "Act 1: Query for Acme Corp").
- **Observe:** The LLM processes the results of its action and summarizes its findings ("Obs 1").

Iteration: The Thought-Act-Observe cycle repeats until the LLM arrives at a satisfactory answer.

Benefits: Enables LLMs to access and leverage external information, leading to more informed and reliable responses

ReAct Prompting

ITERATION 2:

ReAct (Reason + Act)

Thought 2: I got the required information about the billing status of Acme Corps. I need to search for the **account manager** next.

Act 2: `query_crm` [account=Acme Corp]

Obs 2: manager = Priya S.

*The free-form thoughts are used to achieve different tasks such as **decomposing questions**, **extracting information**, **guiding search formulation**, and **synthesizing final answer***

ITERATION 3:

ReAct (Reason + Act)

Thought 3: I have the information about the billing status of Acme Corps as well as the account manager. So the answer is 14 days past due with an amount of ₹1,20,000 and the manager is Priya S.

Act 3: `Finish` [14 days past due; ₹1,20,000 outstanding, AM: Priya S.]

After getting the desired solution the **process terminates** at the **subsequent Act (here Act 3)** stage

Meta Prompting



Think about a whole class of problems, not just one problem

Zero-shot

Only the task - no examples, no reasoning, simple tasks

Few-shot

3-5 solved examples. Reasoning based on only the examples

Chain-of-thoughts

“Think step by step” Steps are not well defined

Meta-prompting

Gives the model a
structure instead of
content

Meta Prompting

Meta Prompting focuses on the **formal structure of a task** rather than on **content-specific examples**

Characteristics

Structure-oriented:

Prioritizes the format and pattern of problems and solutions over specific content.

Syntax-focused: Uses syntax (e.g. `\boxed{}`) as a guiding template for the expected response or solution.

Versatile: Applicable across various domains, capable of providing structured responses to a wide range of problems.

Few-shot

Example 1 -> answer

Example 2 -> answer

Question

Meta-prompt

One reusable template for every question of this type

Advantages:

- **Clarification:** Offers clarity over user's prompt
- **Simplification:** The prompts focus on the essential content and simplify it
- **Accuracy:** Better accuracy because of clear and concise instructions.

Meta Prompting

Example 2 - business metric

“Diagnose an unexpected movement in an business metric and recommend an action.”

Why did EMEA trial conversion drop?

This is just one instance. The template is independent of the instance.

“Act as a business analyst. Diagnose the metric movement described below. Use this structured template:”

1. Restate the metric, the observed change, the time window, and the baseline being compared against.
2. Establish the change is real before explaining it. Check for instrumentation or definition changes, known seasonality, and whether the move exceeds normal period-over-period variance.
3. Decompose the metric into its arithmetic components (e.g. revenue = traffic × conversion × ARPU). Identify which component actually moved.
4. Segment that component across available dimensions — channel, geography, cohort, platform, plan tier. State whether the move is broad-based or concentrated.
5. Generate at least three candidate causes for the concentrated segment. For each, name the evidence that would confirm it and the evidence that would rule it out.
6. Rank candidates by strength of evidence. State explicitly which data you do not have.
7. Recommend one action, its rough cost, and the leading indicator that would show within 14 days whether it worked.
8. Summarize as: what moved / where / most likely why / what to do / confidence level.

Meta Prompting

Example 2 - business metric

Example prompts:

- Trial-to-paid conversion fell 22% -> 17% in EMEA over three weeks.
- Support ticket volume is up 40% month-over-month
- Enterprise renewal rate slipped 4 points this quarter.

“Act as a business analyst. Diagnose the metric movement described below. Use this structured template:”

1. Restate the metric, the observed change, the time window, and the baseline being compared against.
2. Establish the change is real before explaining it. Check for instrumentation or definition changes, known seasonality, and whether the move exceeds normal period-over-period variance.
3. Decompose the metric into its arithmetic components (e.g. revenue = traffic × conversion × ARPU). Identify which component actually moved.
4. Segment that component across available dimensions — channel, geography, cohort, platform, plan tier. State whether the move is broad-based or concentrated.
5. Generate at least three candidate causes for the concentrated segment. For each, name the evidence that would confirm it and the evidence that would rule it out.
6. Rank candidates by strength of evidence. State explicitly which data you do not have.
7. Recommend one action, its rough cost, and the leading indicator that would show within 14 days whether it worked.
8. Summarize as: what moved / where / most likely why / what to do / confidence level.

Meta Prompting

Practical guide

When to use:

- There is a whole category of similar tasks, not just one or two
- The correct reasoning procedure is known
- Consistency and explainability is priority

When not to:

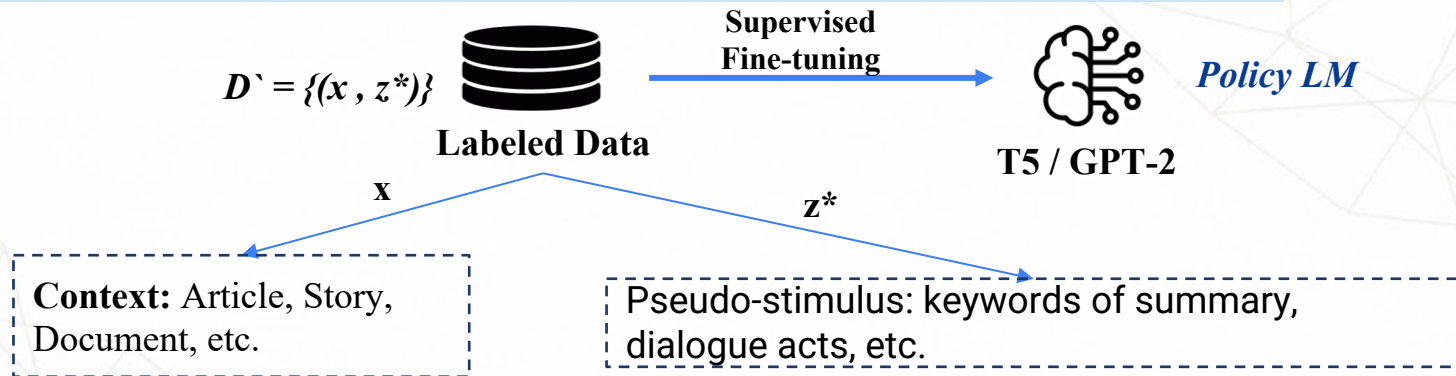
- One-off questions where a prompt is cheaper
- Domains where the right steps cannot be articulated

Directional Stimulus Prompting (DSP)

- The framework introduces a new component called the “directional stimulus” into the prompt to provide nuanced, instance-specific guidance and control over LLMs.
- The directional stimulus prompt acts as “hints” and “clues” for the input query to guide LLMs toward the desired output.
- Notably, this differs from the methods that augment LLMs with additional knowledge retrieved from external sources, as the directional stimulus prompt is generated solely based on the input query in our framework.
- A tuneable policy LM is trained to generate the hints that guide a black-box frozen LLM.
- The approach incorporates keywords in the prompt as the directional stimulus prompt to hint at key points the desired summary should cover.

Directional Stimulus Prompting (DSP)

Step 1: Training a Policy Model that generates directional stimulus for LLMs



Example:

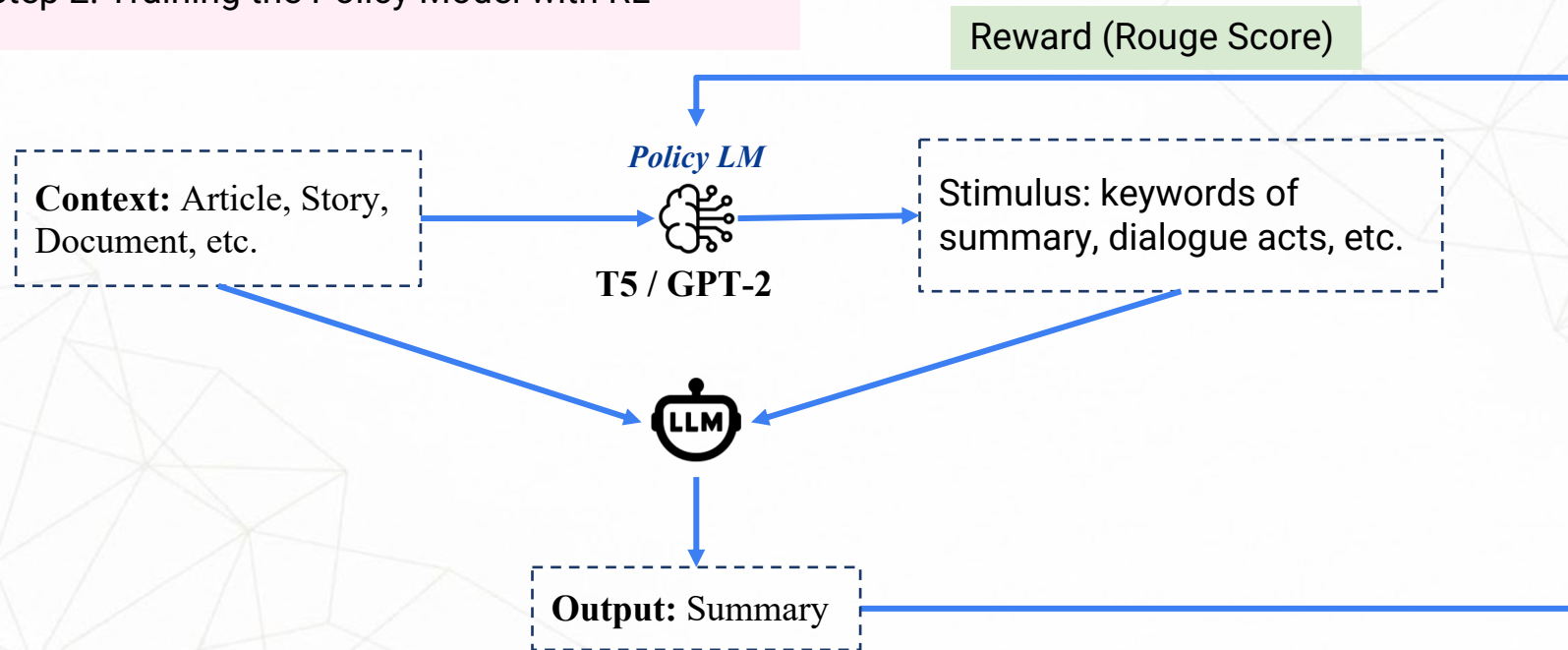
Article: Seoul (CNN) South Korea's Prime Minister Lee Wan-koo offered to resign on Monday amid a growing political scandal. Lee will stay in his official role until South Korean President Park Geun-hye accepts his resignation. He has transferred his role of chairing Cabinet meetings to the deputy prime minister for the time being, according to his office.CNN's Paula Hancocks contributed to this report.

Keywords: Lee Wan-koo; resign, South Korean tycoon; Sung Woan-jong; hanging from a tree; investigation; notes; top officials

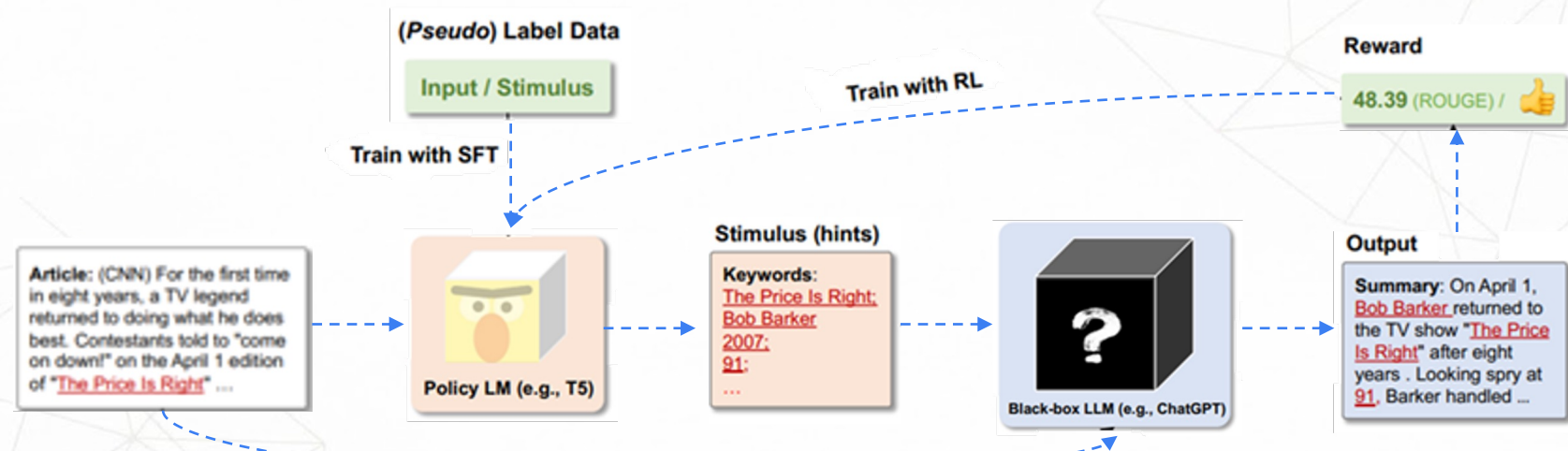
Directional Stimulus Prompting (DSP)



Step 2: Training the Policy Model with RL



Directional Stimulus Prompting (DSP)



Directional Stimulus Prompting (DSP)

Input text

Article: (CNN) For the first time in eight years, a TV legend returned to doing what he does best. Contestants told to "come on down!" on the April 1 edition of **"The Price Is Right"** encountered not host Drew Carey but another familiar face in charge of the proceedings. Instead, there was **Bob Barker**, who hosted the TV game show for 35 years before stepping down in **2007**. Looking spry at **91**, Barker handled the first price-guessing game of the show, the classic "Lucky Seven," before turning hosting duties over to Carey, who finished up. Despite being away from the show for most of the past eight years, Barker didn't seem to miss a beat.

Reference

Bob Barker returned to host **"The Price Is Right"** on Wednesday . Barker, **91**, had retired as host in **2007** .

Standard Prompting

Q: Summarize the above article briefly in 2-3 sentences.

Model output

A: Bob Barker, the former host of "The Price Is Right", returned to the show for the first time in 8 years. He hosted the first game of the show, and then handed the hosting duties over to Drew Carey.

ROUGE-1: 34.48 👎

Directional Stimulus Prompting

Q: Summarize the above article briefly in 2-3 sentences **based on the hint.**

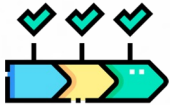
Hint: Bob Barker; TV; April 1: "The Price Is Right"; 2007; 91.

Model output

A: On **April 1**, **Bob Barker** returned to the **TV** show **"The Price Is Right"** after eight years . Looking spry at **91**, Barker handled the first price-guessing game . Barker stepped down as host in **2007**, but didn't seem to miss a beat .

ROUGE-1: 48.39 👍

Prompt Chaining



Decompose one big task into a sequence of smaller, focused LLM calls.

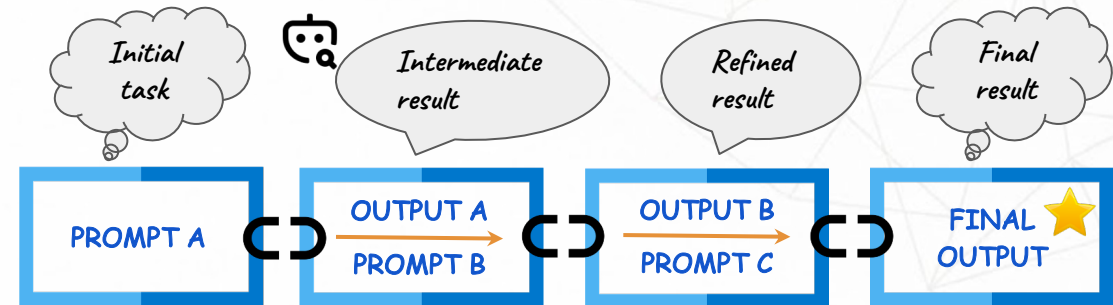
WHAT IT IS

A workflow pattern where the output of one LLM call becomes the input to the next.

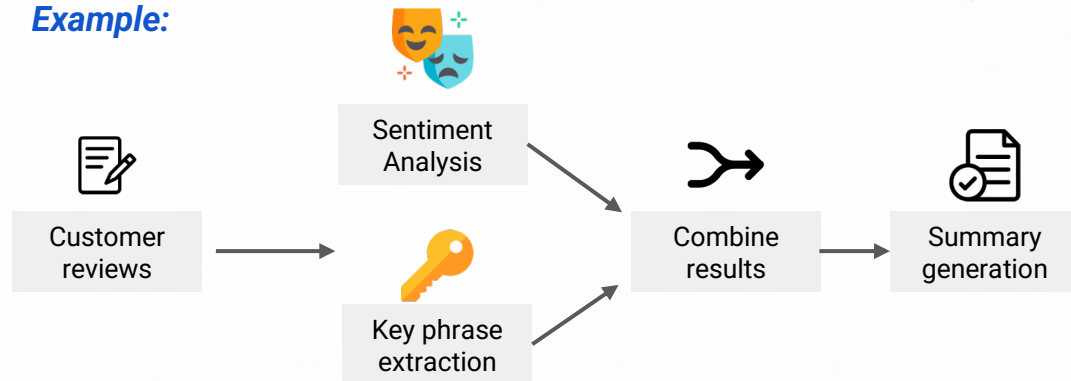
Each step does one thing well like improving accuracy, traceability, and control over complex tasks.

WHEN TO USE

- Task has clean, sequential subtasks
- Each step's output is verifiable
- You want intermediate checkpoints
- Latency budget allows multiple calls



Example:



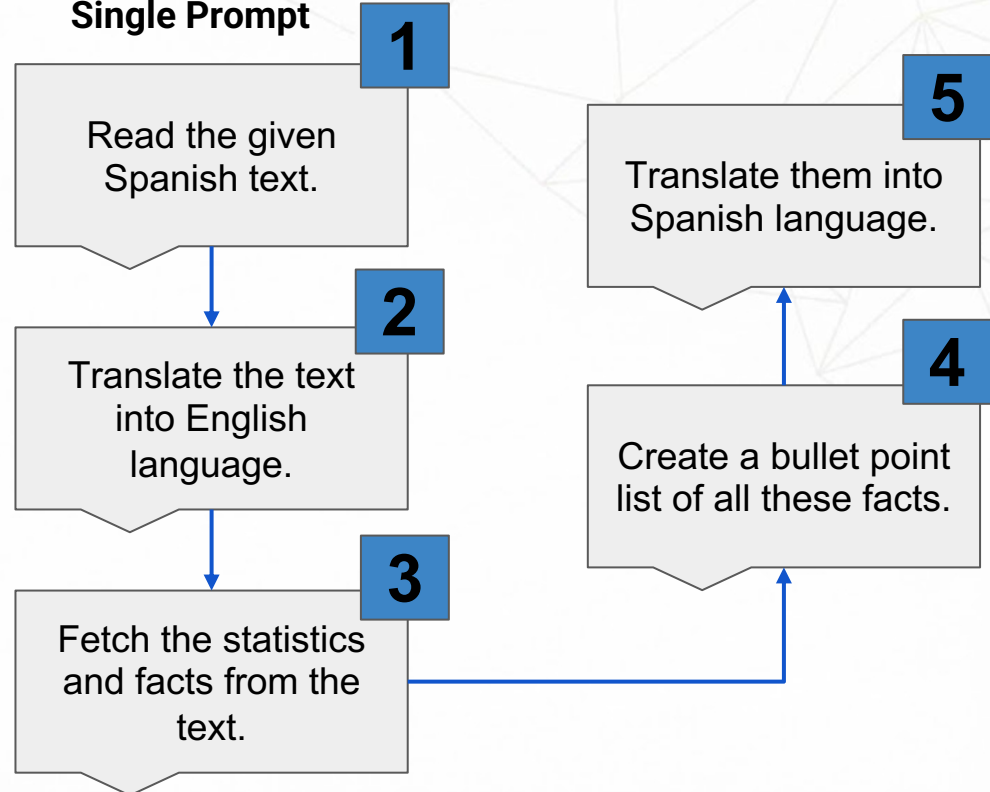
Prompt Chaining

Prompt chaining is useful to accomplish complex tasks which an LLM might struggle to address if prompted with a very detailed prompt.

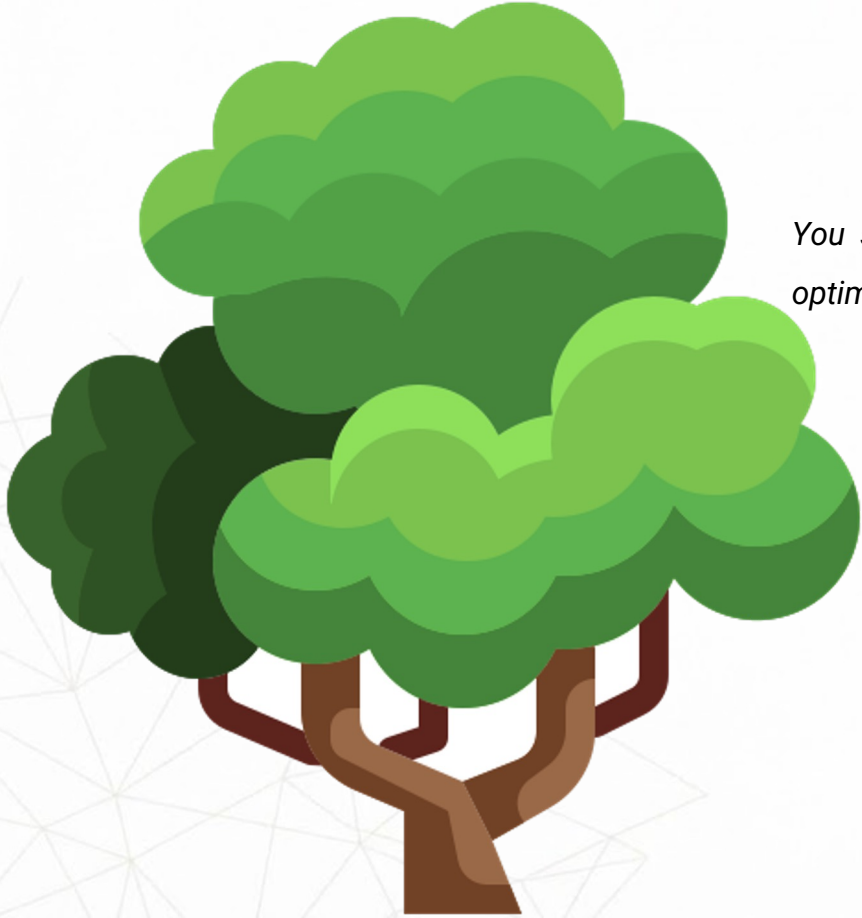
Complex Prompt

Consider the given text in Spanish. Translate it into English. Find all the statistics and facts used in this text and list them as bullet points. Translate them again into Spanish.

Single Prompt



Tree of thoughts (ToT) Prompting



*You should use this for planning tasks, puzzles, creative writing, math optimization, or any multi-step reasoning **where dead ends are common**.*

It shines on problems where the first idea isn't always best

Tree of thoughts (ToT) Prompting



Example

Puzzle: Bob is in the living room. He walks to the kitchen, carrying a cup. He puts a ball in the cup, turns the cup upside down, then carries the cup to the bedroom. Where is the ball?

Usual output [ZERO SHOT]: Puzzle: The ball is in the bedroom.

Tree of thoughts:

Imagine three different experts are answering this question. They will track the ball step-by-step, evaluate the physics of each action, and branch out to cover different possible interpretations.

EXPERT 1 (Physics)

Cup upside down, gravity makes ball fall out in kitchen

Conclusion: Kitchen floor

EXPERT 2 (Language)

Story never says ball stayed inside the cup. "Upside down" implies the ball dropped before moving rooms.

Conclusion: Kitchen

EXPERT 3 (Skeptic)

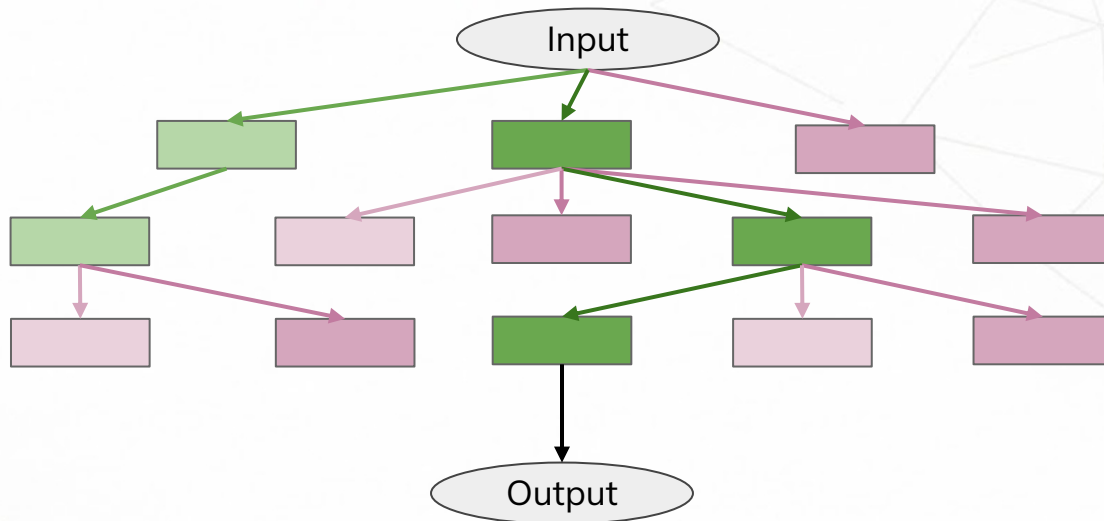
The text does not specify the cup type or the ball size. If the ball is small and lodged in the rim, stuck by suction, or the cup has a hidden lid, it could remain inside while Bob carries it to the bedroom. The narrative leaves that possibility open.

Conclusion: Bedroom (ball still in cup)

FINAL CONSENSUS [Response]

The ball is in the kitchen.

Tree of thoughts (ToT) Prompting



In Tree of Thoughts (ToT) framework, we organize the LLM’s thoughts into nodes in a ToT fashion i.e. combining thought with a symbolic tree search algorithm.

Principle: Tree of Thought prompting is a more sophisticated approach where multiple lines of reasoning are considered in parallel. It's like creating a decision tree

Tree of thoughts (ToT) Prompting

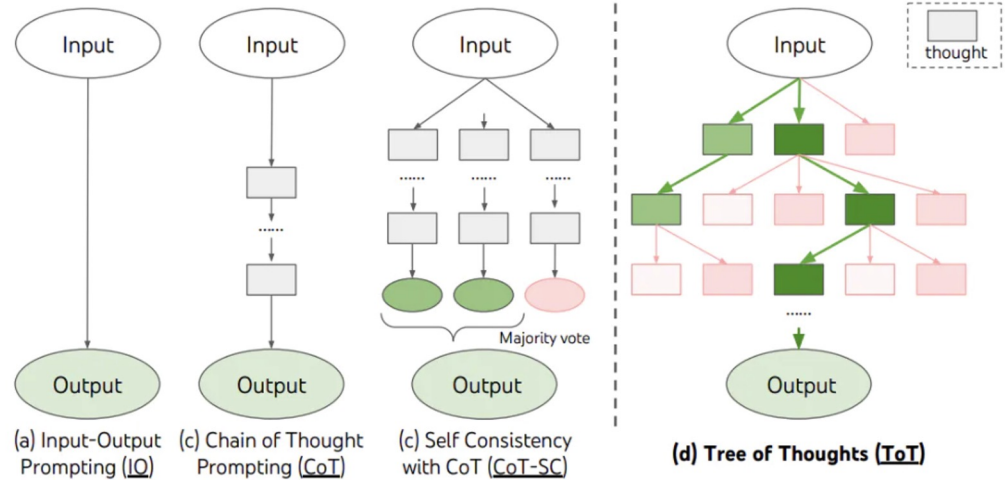
Key Features:

Multiple Reasoning Paths: Explores various lines of thought, unlike linear approaches.

Self-Evaluation: The LLM assesses the quality and progress of its own generated thoughts.

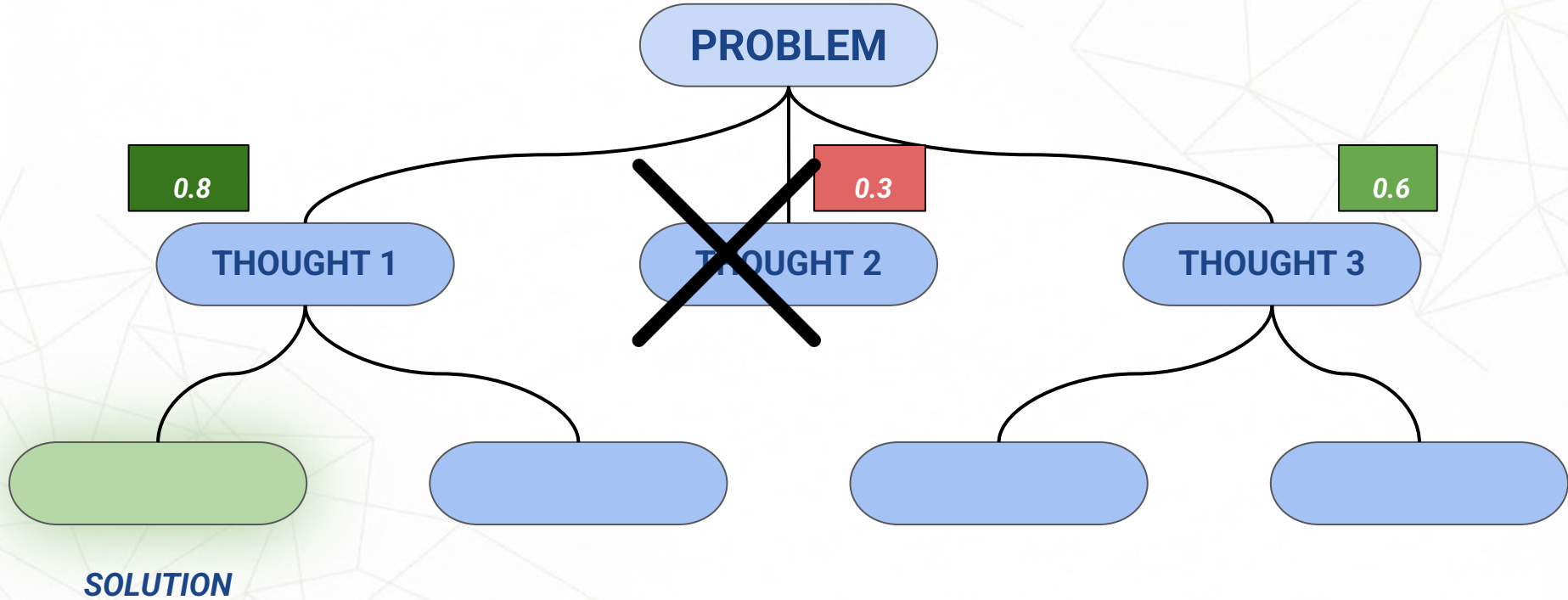
Search Algorithms: Uses techniques like breadth-first or depth-first search to efficiently explore the thought tree, enabling lookahead and backtracking.

Use Cases: Ideal for problems with multiple valid approaches, branching possibilities, or complex decision-making.



Performance: Demonstrates significant improvements over other prompting methods, achieving state-of-the-art results in various reasoning tasks.

Tree of thoughts (ToT) Prompting



Limitations of LLM Prompting

Hallucination

- Hallucination refers to a response from an LLM that includes false or misleading information presented as a fact.



What is the natural habitat of the Elysian Phoenix butterfly?



Answer : The Elysian Phoenix butterfly (Elymnias hypermnestra) is a species of brush-footed butterfly found in the Indo-Australian region



Correct Answer : The "Elysian Phoenix butterfly" does not exist; the model fabricated the species and habitat.



Tell me about the 2007 earthquake in San Diego.



Answer : The 2007 San Diego earthquake, also known as the "Borrego Mountain earthquake," occurred on April 4, 2007, at 10:43 AM local time



Correct Answer : "Borrego Mountain earthquake" happened in 1968. There was no major earthquake in San Diego in 2007

Mitigating prompt-injection attacks

Mistral 7B Guardrails

- System prompts don't just tell LLMs what to do. They also include safeguards that tell the LLM what not to do.

For example, a simple translation app's system prompt might read:

You are a translation chatbot. You do not translate any statements containing profanity. Translate the following text from English to French:

These safeguards aim to stop people from using LLMs for unintended actions—in this case, from making the bot say something offensive.

- The model can enforce guardrails for AI generation, ensuring that the generated content is safe and adheres to specified guidelines. **For Mistral**, The system prompt recommended to enforce guardrails is as follows:

Always assist with care, respect, and truth. Respond with utmost utility yet securely. Avoid harmful, unethical, prejudiced, or negative content. Ensure replies promote fairness and positivity.

Thank You

Instructor



Koustav Rudra
krudra@ai.iitkgp.ac.in

Teaching Assistants (IITKGP)



Anurag Sharma



Subhojyoti Khastagiri



Srinjoy Chanda